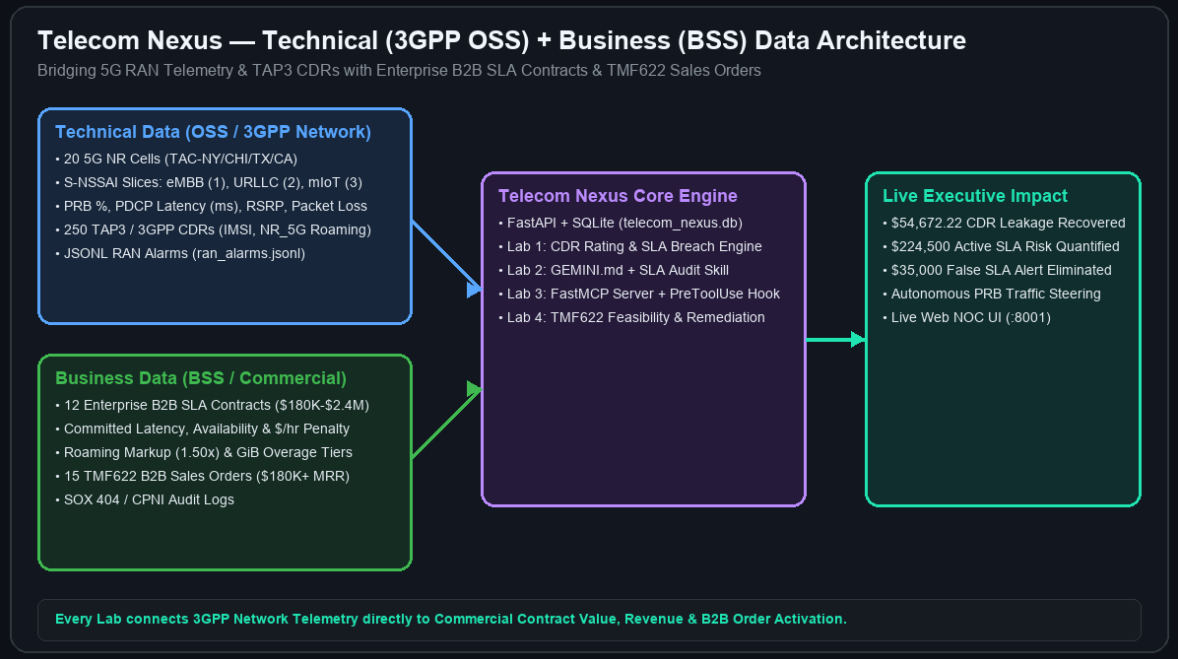
Agentic Engineering with Google Antigravity — Telecom Edition: Building 'Telecom Nexus'

5G Network Slice SLA, CDR Revenue Assurance & B2B Contract Intelligence (Delivered by Tryolabs)



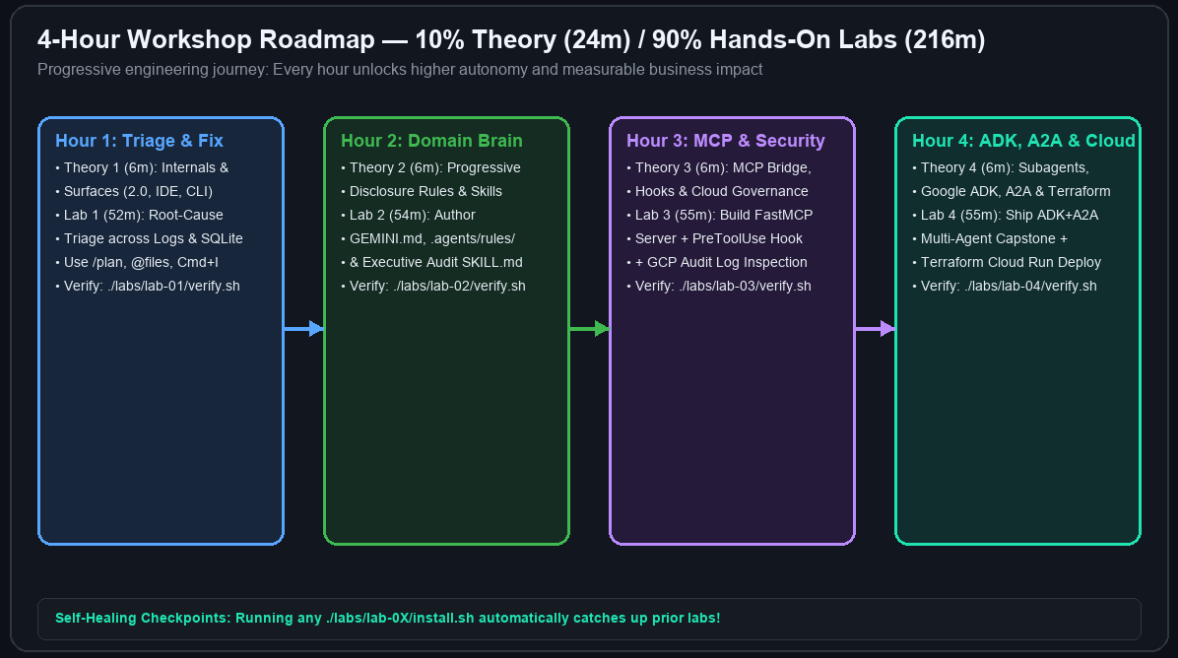
Workshop Mission & Industry Architecture

- **OSS Technical Data:** 20 5G NR cells (gNodeB, TAC, S-NSSAI SST 1/2/3, PRB%, PDCP latency)
- **TAP3 / 3GPP CDRs:** 250 call/data records (IMSI, MSISDN, NR_5G roaming PLMN, GiB volume)
- **BSS Business Data:** 12 Enterprise B2B SLA contracts ($180K–$2.4M ACV, $/hr SLA penalties)
- **TMF622 Sales Pipeline:** 15 pending B2B slice orders + JSONL RAN alarm telemetry
- **Tangible Impact:** Recover $54,672.22 in unbilled CDRs & protect $224,500 in active SLA exposure

tryo•labs

4-Hour Hands-On Roadmap: From Triage to Multi-Agent ADK & A2A

Four 6-minute micro-theory briefings paired with four progressive 52–55 minute hands-on labs



Minute-by-Minute Workshop Schedule

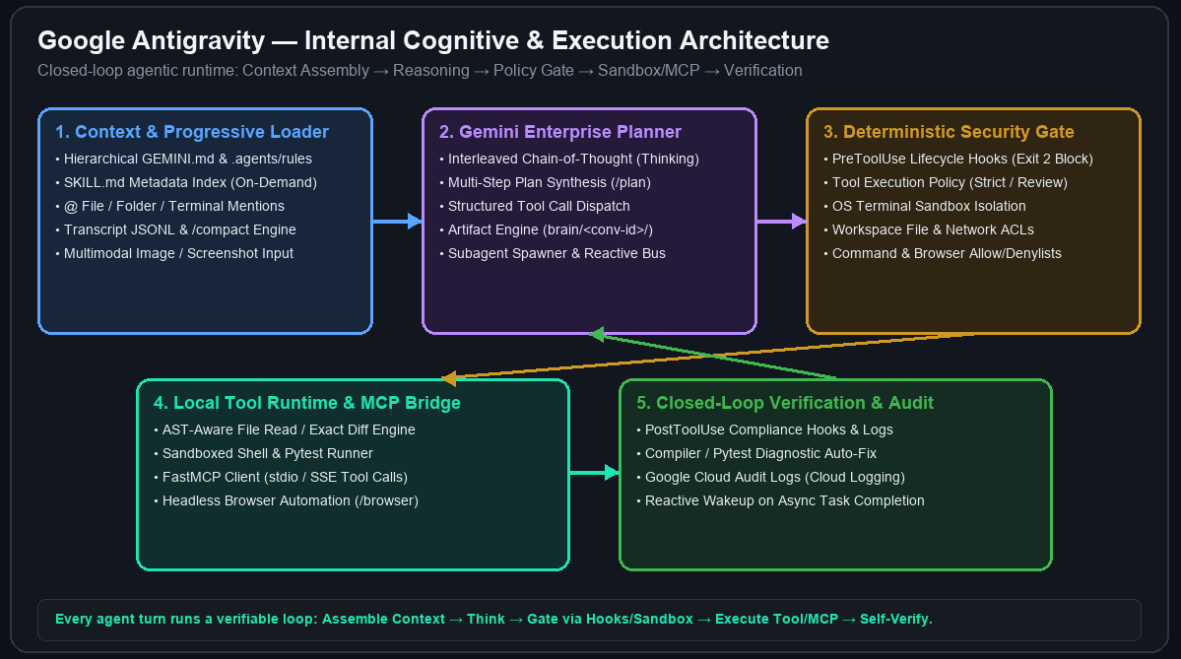
- **Hour 1 (00:00–00:58):** [6m Theory] Internal Architecture & Surfaces → [52m Lab 1] Triage & Revenue/Margin Recovery
- **Hour 2 (00:58–01:58):** [6m Theory] Rules & Progressive Skills → [54m Lab 2] Domain Standards & Executive Risk Skill
- **Hour 3 (01:58–02:59):** [6m Theory] MCP, Hooks & Cloud Governance → [55m Lab 3] Local FastMCP Server & Compliance Hooks
- **Hour 4 (02:59–04:00):** [6m Theory] Subagents, ADK, A2A & Terraform → [55m Lab 4] Multi-Agent Capstone, ADK Dev UI & Cloud Deploy
- **Self-Healing Checkpoints:** Running ./labs/lab-0X/install.sh automatically applies prior lab solutions!

Every lab includes LAB_0X_INSTRUCTIONS_EN.pdf, README.md, PROMPTS.md, install.sh, and verify.sh. | Slide 2/15

tryo•labs

How Google Antigravity Works Internally

Inside the closed-loop cognitive planner, context assembler, policy gate, and sandboxed runtime



The 5-Stage Internal Execution Loop

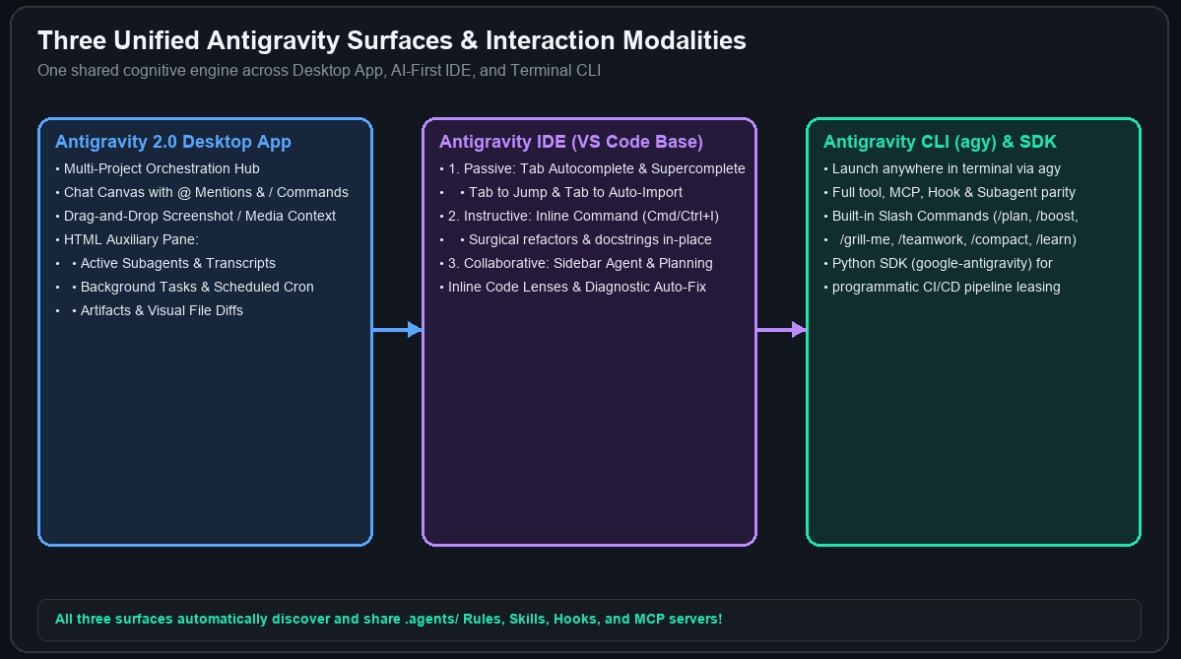
- **1. Context & Progressive Loader:** Walks directory tree for GEMINI.md & .agents/rules, indexes SKILL.md YAML frontmatter, resolves @mentions, and manages transcript.jsonl
- **2. Gemini Enterprise Planner:** Generates interleaved chain-of-thought (thinking), synthesizes plans (/plan), and dispatches parallel tool or subagent calls
- **3. Deterministic Security Gate:** Evaluates PreToolUse hooks (exit code 2 blocks execution), tool policies, file/network ACLs, and OS sandbox rules
- **4. Tool Runtime & MCP Bridge:** Executes AST-aware file diffs, sandboxed pytest commands, and stdio/SSE FastMCP calls
- **5. Verification & Reactive Loop:** Runs PostToolUse hooks, reads compiler/test outputs, and self-corrects until green

Key Insight: Antigravity is not just chat—it is a stateful, verifiable closed-loop engineering runtime. | **Slide 3/15**

tryo•labs

Antigravity 2.0, IDE & CLI (agy) — Core Modalities

Choosing the right modality and slash command for every engineering task



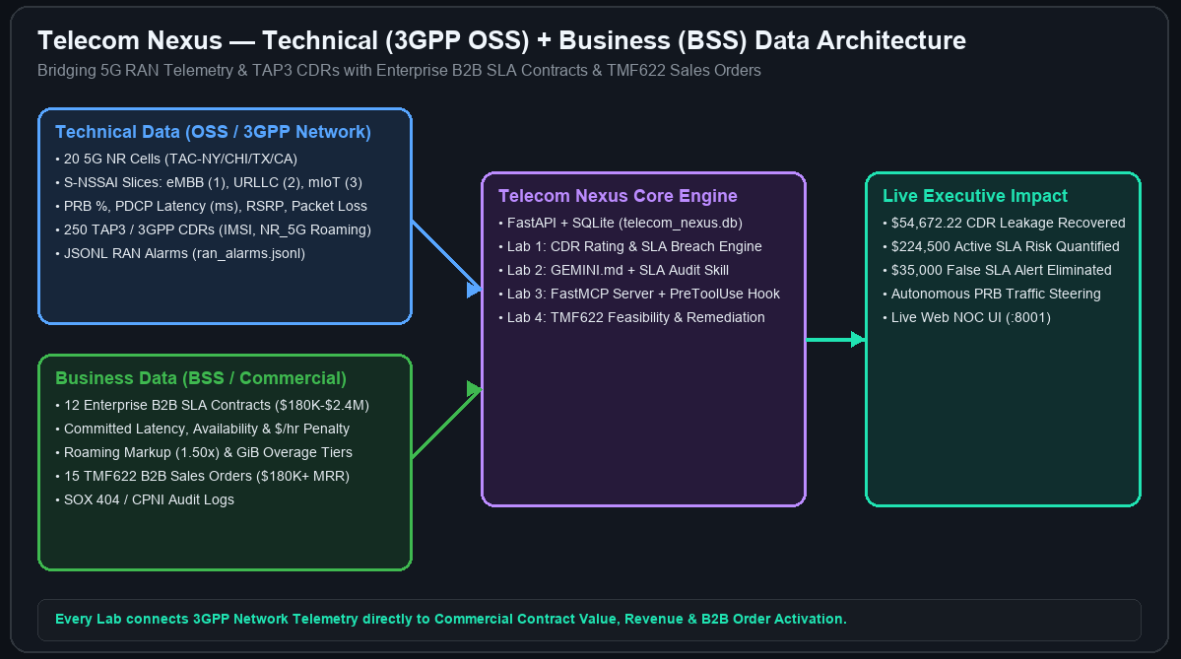
Features You Will Explore Today

- **Passive Modality (IDE Tab):** Context-aware Supercomplete, Tab-to-Jump across files, and Tab-to-Auto-Import
- **Instructive Modality (Cmd/Ctrl+I & Code Lenses):** Highlight a function for surgical inline refactoring or click 'Write Tests'
- **Collaborative Modality (Chat Canvas & CLI agy):** Multi-file reasoning with @file, @folder, @terminal, and screenshot drag-and-drop
- **High-Leverage Slash Commands:** /plan (step-by-step architecture), /grill-me (design interview), /boost, /teamwork, /learn & /compact
- **HTML Auxiliary Pane:** Real-time inspection of Subagents, Background Tasks, Artifacts, and File Diffs

Pro Tip: Start complex tasks with /plan and @file mentions; use Cmd+I for localized formula fixes.

Lab 1: 5G Slice SLA & CDR Revenue Triage — Objectives & Build Spec

52 Minutes • Printable PDF Guide: labs/lab-01-revenue-sla-triage/LAB_01_INSTRUCTIONS_EN.pdf



Lab 1 Objectives & What You Will Build

- **Objective 1 (Multi-File Agentic Triage):** Master Antigravity `@mentions`, `/plan`, and inline `Cmd/Ctrl+I` to correlate 3GPP RAN alarms (`ran_alarms.jsonl`) with BSS billing & SLA code.
- **Objective 2 (Revenue & SLA Accuracy):** Eliminate floating-point/unit bugs leaking `$54,672.22` in 5G CDRs and stop `$35,000` in false B2B SLA alerts on non-active contracts.
- **What You Will Build #1 (3GPP TAP3 Rating Engine):** Refactor `src/services/cdr_rating.py` to enforce binary GiB (`Decimal(1024 ** 3)`) and apply `roaming_markup` across all visiting PLMNs (`NR_5G` + `EUTRAN_4G`).
- **What You Will Build #2 (B2B SLA Breach Monitor):** Refactor `src/services/sla_monitor.py` to filter `WHERE status = 'ACTIVE'` and trigger breaches when `worst_latency > committed_latency`.
- **Deliverable & Verification:** Pass all 5 unit/integration tests in `tests/test_lab01_triage.py` (`./labs/lab-01-revenue-sla-triage/verify.sh`) and verify KPIs on `http://localhost:8001`.

tryo•labs

Lab 1 Instructions & Copy-Paste Antigravity Prompts

Follow Steps 1–5 on the left and copy-paste prompts from labs/lab-01-revenue-sla-triage/PROMPTS.md

Step-by-Step Lab 1 Instructions

- **Step 1 (Initialize Lab 01):** Run `./labs/lab-01-revenue-sla-triage/install.sh` to seed `data/telecom_nexus.db` and activate the starter bugs.
- **Step 2 (Reproduce Failing Tests):** Run `./.venv/bin/pytest tests/test_lab01_triage.py` to observe the CDR underbilling and SLA latency failures.
- **Step 3 (Fix `src/services/cdr_rating.py`):** Set `BYTES_DIVISOR = Decimal(1024 ** 3)` (binary GiB per 3GPP TS 32.298) and apply `roaming_markup` whenever `roaming_plmn != HOME_PLMN`.
- **Step 4 (Fix `src/services/sla_monitor.py`):** Add `WHERE status = 'ACTIVE'` to the SQL query and fix `latency_breach` to `worst_latency > committed_latency`.
- **Step 5 (Verify & Inspect):** Run `./labs/lab-01-revenue-sla-triage/verify.sh` (`5 passed`: `$54,672.22` CDR leakage recovered; 5 `ACTIVE` SLA breaches = `$224,500.00`).

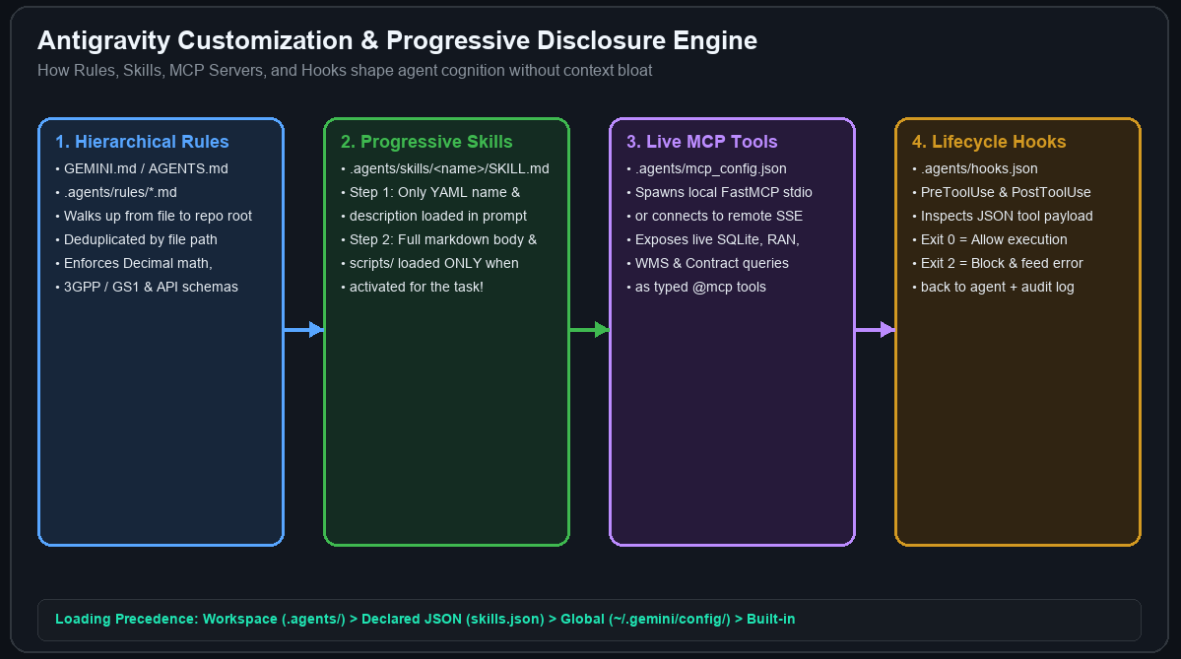
Copy-Paste Prompts (PROMPTS.md) & Verification

- **Prompt 1 — Discovery (@Mentions):** '@data/logs/ran_alarms.jsonl @src/services/cdr_rating.py @src/services/sla_monitor.py @tests/test_lab01_triage.py Run pytest to reproduce ALM-BSS-8006 and ALM-3GPP-8001..8005 and explain the exact root causes.'
- **Prompt 2 — Planning (/plan):** '/plan Create a step-by-step remediation plan to fix cdr_rating.py (10243 GiB & NR_5G roaming markup) and sla_monitor.py (ACTIVE contracts & worst_latency > committed_latency).'
- **Prompt 3 — Inline Edit (Cmd+I):** Select lines 19–48 in `cdr_rating.py` -> 'Fix BYTES_DIVISOR to Decimal(10243) and apply roaming markup to all visiting PLMNs including NR_5G.'
- **Prompt 4 — Autonomous Verify:** 'Apply fixes to @src/services/cdr_rating.py and @src/services/sla_monitor.py, then run ./labs/lab-01-revenue-sla-triage/verify.sh.'

tryo•labs

How Rules & Progressive Disclosure Skills Work Internally

Specializing Antigravity for your industry without overflowing the context window



Rules vs. Skills vs. /learn

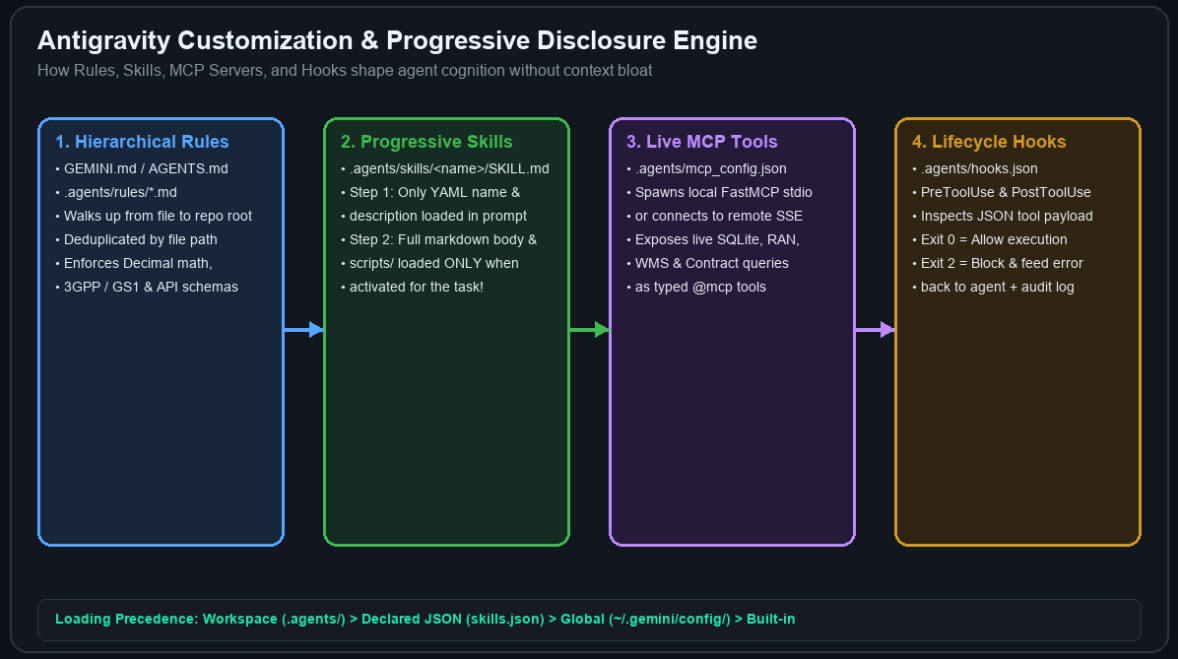
- **Hierarchical Rules (GEMINI.md & .agents/rules/*.md):** Always-on or model-decided domain invariants loaded from workspace root down to target directories (deduplicated automatically)
- **Progressive Disclosure Skills (.agents/skills/<name>/SKILL.md):** Only lightweight YAML frontmatter (name, description) sits in system prompt; full markdown runbook & scripts/ load ONLY when triggered!
- **Self-Improving Memory (/learn):** Ask Antigravity to codify a newly discovered debugging pattern or architectural standard into a reusable workspace customization
- **Precedence Hierarchy:** Workspace (.agents/) > Declared JSON (skills.json) > Global (~/.gemini/config/) > Built-in

Rule of Thumb: Put permanent guardrails in GEMINI.md/rules; put multi-step workflows & scripts in SKILL.md.

tryo•labs

Lab 2: Workspace Rules & Progressive Skills — Objectives & Build Spec

54 Minutes • Printable PDF Guide: labs/lab-02-rules-and-skills/LAB_02_INSTRUCTIONS_EN.pdf



Lab 2 Objectives & What You Will Build

- **Objective 1 (Hierarchical Domain Rules):** Teach Antigravity permanent 3GPP, TAP3, TMF622, and CPNI invariants so every future prompt obeys telecom standards automatically.
- **Objective 2 (Progressive Disclosure Skills):** Package a repeatable executive audit workflow into `.agents/skills/` where only YAML frontmatter loads until triggered on demand.
- **What You Will Build #1 (`GEMINI.md` & `.agents/rules/telecom-standards.md`):** Workspace memory & rule files enforcing `Decimal(ROUND_HALF_UP)`, binary GiB (`1024**3`), `PRB < 80%`, and CPNI `IMSI`/`MSISDN` masking.
- **What You Will Build #2 (`.agents/skills/sla-revenue-audit/SKILL.md`):** Progressive disclosure skill with YAML frontmatter (`name`, `description`) that runs `scripts/run_sla_audit.py`.
- **Deliverable & Verification:** Generate `data/exports/sla_revenue_audit_report.json` & `.md` and pass all 4 tests in `./labs/lab-02-rules-and-skills/verify.sh`.

tryo•labs

Lab 2 Instructions & Copy-Paste Antigravity Prompts

Follow Steps 1–5 on the left and copy-paste prompts from labs/lab-02-rules-and-skills/PROMPTS.md

Step-by-Step Lab 2 Instructions

- **Step 1 (Initialize Lab 02):** Run `./labs/lab-02-rules-and-skills/install.sh` (automatically applies Lab 01 solution first).
- **Step 2 (Author Root `GEMINI.md`):** Define OSS/BSS tables (`ran_telemetry`, `b2b_contracts`, `cdr_records`), `Decimal(ROUND_HALF_UP)` billing math, and `1024 ** 3` GiB.
- **Step 3 (Author `.agents/rules/telecom-standards.md`):** Codify 3GPP TS 28.552 (`PRB < 80%`), TAP3 `NR_5G` roaming rules, TMF622 order states, and CPNI `IMSI`/`MSISDN` masking.
- **Step 4 (Author `.agents/skills/sla-revenue-audit/SKILL.md`):** Add YAML frontmatter (`name: sla-revenue-audit`, `description: ...`) and instructions to run `./.venv/bin/python scripts/run_sla_audit.py`.
- **Step 5 (Verify):** Run `./labs/lab-02-rules-and-skills/verify.sh` (`4 passed` + executive JSON/MD reports generated).

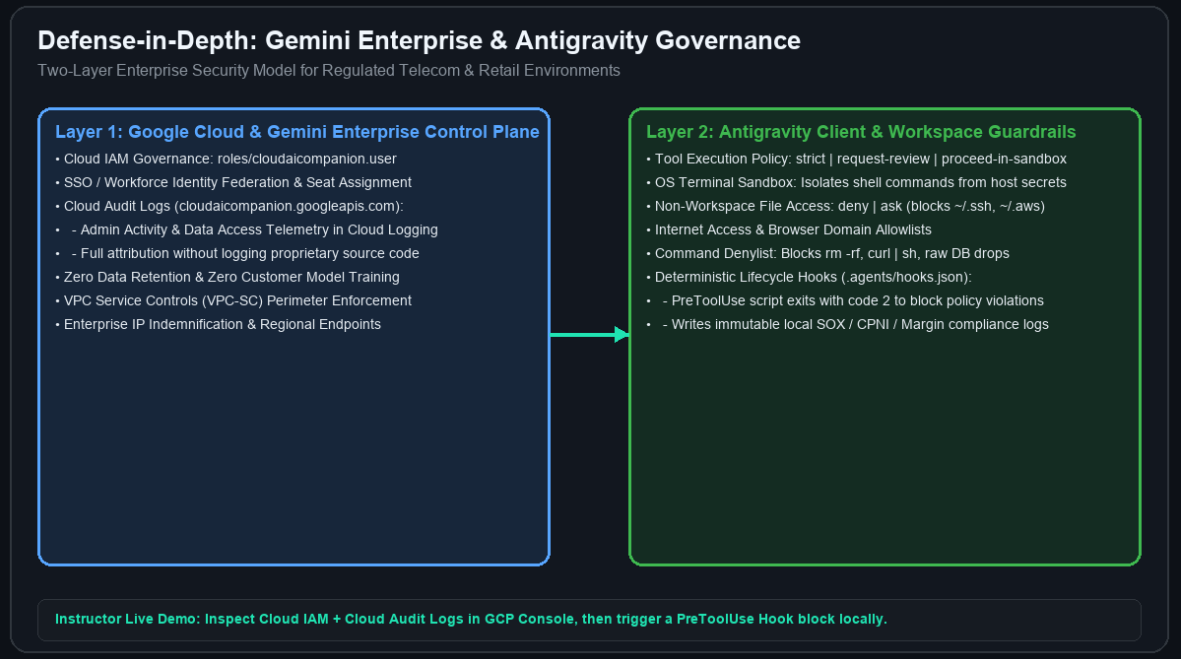
Copy-Paste Prompts (PROMPTS.md) & Verification

- **Prompt 1 — Author Workspace Rules:** '/plan Create @GEMINI.md and @.agents/rules/telecom-standards.md codifying 3GPP TS 28.552 PRB/PDCP thresholds, binary GiB (10243), Decimal ROUND_HALF_UP, TMF622 order states, and CPNI IMSI/MSISDN masking.'
- **Prompt 2 — Build Progressive Skill:** 'Create @.agents/skills/sla-revenue-audit/SKILL.md with YAML frontmatter (name: sla-revenue-audit) that runs ./scripts/run_sla_audit.py and outputs data/exports/sla_revenue_audit_report.json and .md.'
- **Prompt 3 — Trigger Skill On-Demand:** 'Run an executive 5G Slice SLA and CDR revenue assurance audit using our workspace skill and summarize the top breached contracts.'
- **Verify Command:** Run `./labs/lab-02-rules-and-skills/verify.sh` and test `/learn`!

tryo•labs

Live MCP Bridge, Deterministic Hooks & Gemini Enterprise Governance

Connecting live databases safely while proving end-to-end enterprise security in Google Cloud



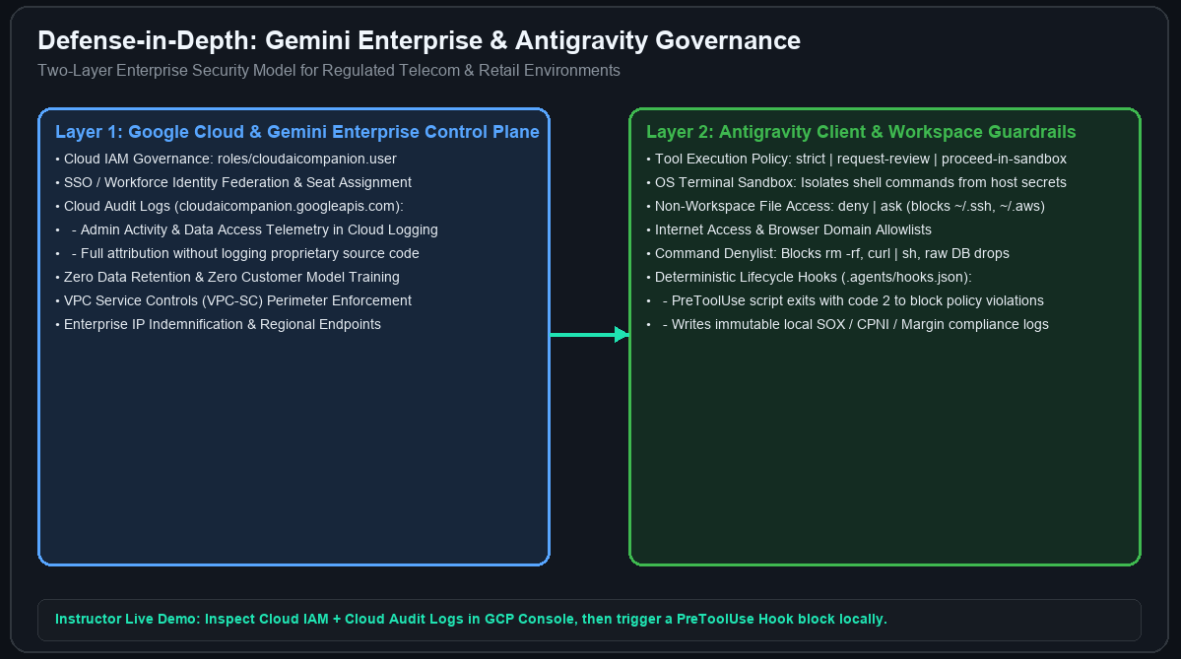
2-Layer Enterprise Security & Governance

- **Layer 1 — Google Cloud & Gemini Enterprise (Instructor Live Demo):** Cloud IAM (roles/cloudaicompanion.user), Cloud Audit Logs (cloudaicompanion.googleapis.com), VPC-SC perimeters, Zero Data Retention / No Training on customer code
- **Layer 2 — Antigravity Client Security:** Tool Execution Policy (strict / request-review / proceed-in-sandbox), OS Terminal Sandbox, Non-Workspace File Access (deny), Command & Browser Allow/Denylists
- **Live Context via FastMCP (.agents/mcp_config.json):** Exposes strongly-typed Python tools querying live SQLite & telemetry over stdio/SSE
- **Deterministic Lifecycle Hooks (.agents/hooks.json):** PreToolUse script inspects tool JSON on stdin; exit code 2 hard-blocks policy violations and writes an immutable audit log!

Instructor Demo: Show Cloud Audit Logs in GCP Console + trigger a deterministic PreToolUse Hook block locally! | **Slide 10/15**

Lab 3: FastMCP Server & Deterministic Hooks — Objectives & Build Spec

55 Minutes • Printable PDF Guide: labs/lab-03-mcp-and-hooks/LAB_03_INSTRUCTIONS_EN.pdf



Lab 3 Objectives & What You Will Build

- **Objective 1 (Live Database Grounding via MCP):** Expose read-only SQLite OSS/BSS telemetry and contract tools to Antigravity over Model Context Protocol (`FastMCP`).
- **Objective 2 (Deterministic Security & CPNI Compliance):** Enforce PII/CPNI masking (`3104***0001`) and hard-block destructive SQL (`DROP`/`DELETE`) before tool execution via `PreToolUse` hooks.
- What You Will Build #1 (`mcp_server/telecom_mcp.py` & `.agents/mcp_config.json`): 4 `@mcp.tool()` functions (`query_ran_cell_health`, `calculate_contract_penalty_exposure`, `inspect_unbilled_cdrs`, `evaluate_slice_capacity`).
- **What You Will Build #2 (`.agents/hooks.json` & `scripts/audit_hook.py`):** Deterministic `PreToolUse` & `PostToolUse` lifecycle gates returning exit code `2` on policy violations and writing `data/logs/hook_audit.log`.
- **Deliverable & Verification:** Pass all 6 MCP & Hook security tests in `./labs/lab-03-mcp-and-hooks/verify.sh`.

tryo•labs

Lab 3 Instructions & Copy-Paste Antigravity Prompts

Follow Steps 1–5 on the left and copy-paste prompts from labs/lab-03-mcp-and-hooks/PROMPTS.md

Step-by-Step Lab 3 Instructions

- **Step 1 (Initialize Lab 03):** Run `./labs/lab-03-mcp-and-hooks/install.sh` (auto-applies Labs 01–02).
- **Step 2 (Complete `mcp_server/telecom_mcp.py`):** Implement `calculate_contract_penalty_exposure`, `inspect_unbilled_cdrs` (masking `imsi`/`msisdn` via `_mask_cpni`), and `evaluate_slice_capacity`.
- **Step 3 (Register `.agents/mcp_config.json`):** Configure `telecom-nexus-mcp` invoking `./.venv/bin/python mcp_server/telecom_mcp.py` over stdio.
- **Step 4 (Configure `.agents/hooks.json`):** Wire `PreToolUse` and `PostToolUse` to `scripts/audit_hook.py` (blocks `DROP`/`TRUNCATE`/`DELETE` with exit code `2`).
- **Step 5 (Verify):** Run `./labs/lab-03-mcp-and-hooks/verify.sh` (`6 passed` + audit log verified).

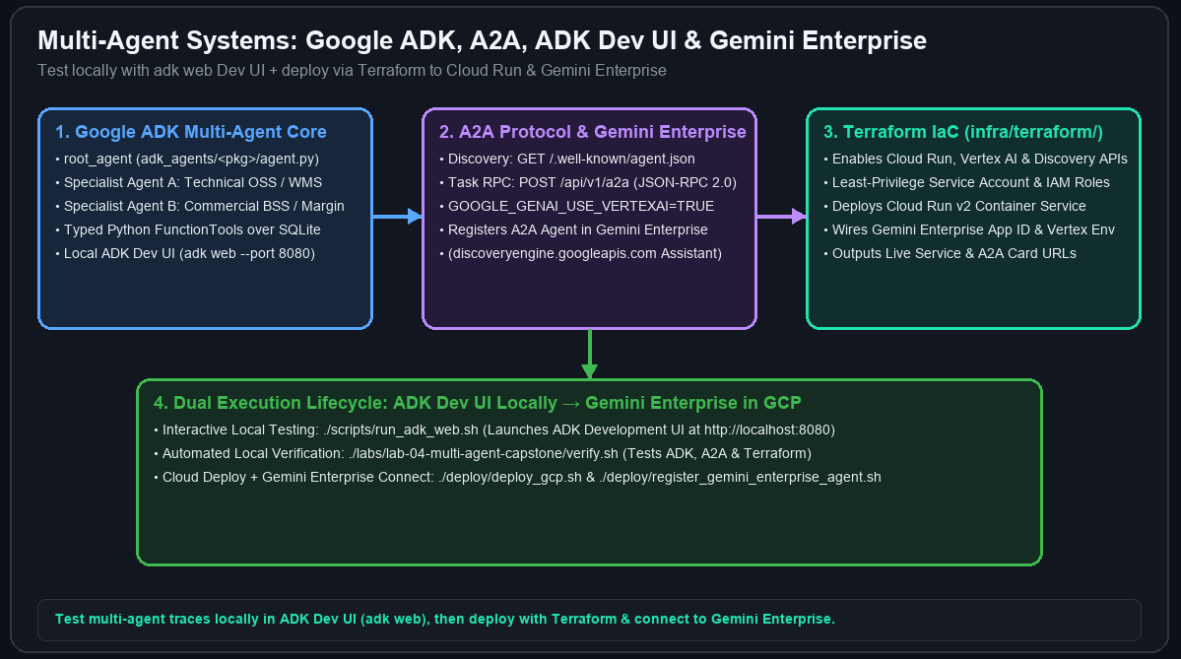
Copy-Paste Prompts (PROMPTS.md) & Verification

- **Prompt 1 — Complete FastMCP Server:** '/plan Implement the 3 remaining @mcp.tool() functions in @mcp_server/telecom_mcp.py (calculate_contract_penalty_exposure, inspect_unbilled_cdrs with CPNI masking, evaluate_slice_capacity) to match @tests/test_lab03_mcp_hooks.py.'
- **Prompt 2 — Wire MCP & Hooks Config:** 'Create @.agents/mcp_config.json for telecom-nexus-mcp and @.agents/hooks.json wiring PreToolUse and PostToolUse to ./scripts/audit_hook.py.'
- **Prompt 3 — Test Hook Block (Exit Code 2):** Run `echo '{"command":"sqlite3 data/telecom_nexus.db \"DROP TABLE b2b_contracts;\""}' | ./.venv/bin/python scripts/audit_hook.py --phase pre`
- **Verify Command:** Run `./labs/lab-03-mcp-and-hooks/verify.sh` and inspect `data/logs/hook_audit.log`.

tryo•labs

Multi-Agent Systems with Google ADK, A2A Protocol & Terraform

Designing multi-agent architectures locally with `adk web` and deploying to Cloud Run & Gemini Enterprise



End-to-End Multi-Agent Architecture

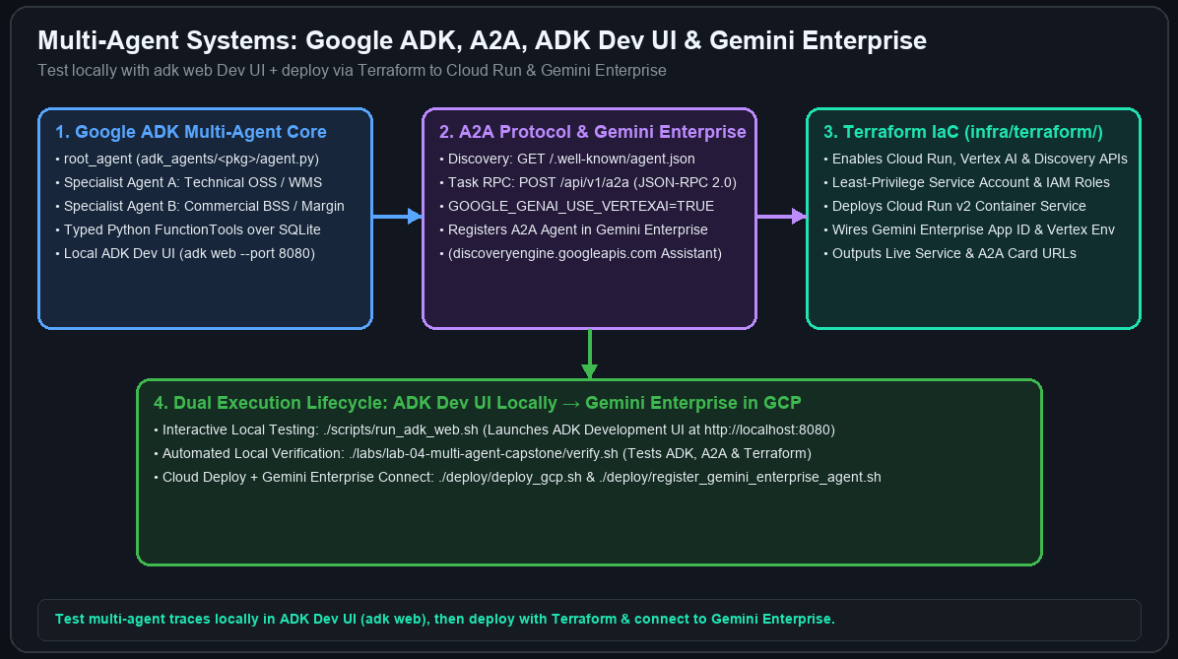
- **Antigravity Subagents (/teamwork, /boost, /grill-me):** Delegate parallel coding & research tasks inside Antigravity to build your application faster
- **Google ADK (Agent Development Kit):** Build domain multi-agent hierarchies (`root_agent` orchestrating Technical & Commercial specialist `LlmAgent`s with typed Python tools)
- **Local Testing via ADK Dev UI (`adk web`):** Launch `./scripts/run_adk_web.sh` (http://localhost:8080) to interactively inspect multi-agent event traces, tool calls, and session state!
- **A2A (Agent2Agent) Protocol:** Publish `GET /.well-known/agent.json` (Agent Card) and `POST /api/v1/a2a` (JSON-RPC 2.0 `tasks/send`) for cross-agent interoperability
- **Terraform + Gemini Enterprise:** Provision Cloud Run v2 & IAM via `infra/terraform/` (`deploy_gcp.sh`) and register your A2A agent into Gemini Enterprise (`register_gemini_enterprise_agent.sh`)

Develop & test 100% locally with `adk web` + `pytest`, then deploy to Google Cloud & Gemini Enterprise with Terraform! | **Slide 13/15**

tryo•labs

Lab 4 Capstone: Multi-Agent ADK, A2A & Terraform — Objectives & Build Spec

55 Minutes • Printable PDF Guide: labs/lab-04-multi-agent-capstone/LAB_04_INSTRUCTIONS_EN.pdf



Lab 4 Objectives & What You Will Build

- **Objective 1 (Multi-Agent System Design with Google ADK & A2A):** Design a coordinator + specialist multi-agent hierarchy using Google ADK (`LlmAgent`) and expose it over the A2A JSON-RPC 2.0 protocol.
- **Objective 2 (Local ADK Dev UI & Cloud Terraform Deployment):** Test multi-agent trajectories locally in `adk web` (`http://localhost:8080`) and provision Cloud Run + Gemini Enterprise registration via Terraform.
- **What You Will Build #1 (`src/services/order_feasibility.py`):** TMF622 5G slice feasibility evaluator (`PRB < 80%`) and autonomous neighbor-cell load steering engine (`remediate_slice_for_order`).
- What You Will Build #2 (`src/agents/adk_a2a_orchestrator.py` & `adk_agents/telecom_nexus/`): `RANSliceCapacityAgent` + `BSSCommercialSLAAgent` coordinated by `TelecomNexusOrchestratorAgent`, plus `GET /.well-known/agent.json` & `POST /api/v1/a2a`.
- **What You Will Build #3 (Cloud Infrastructure):** Validate `infra/terraform/main.tf` and Gemini Enterprise registration via `./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run`.

tryo•labs

Lab 4 Instructions, `adk web` Testing & Terraform Cloud Deploy

Follow Steps 1–5 on the left and copy-paste prompts from labs/lab-04-multi-agent-capstone/PROMPTS.md

Step-by-Step Lab 4 Instructions

- **Step 1 (Initialize Lab 04):** Run `./labs/lab-04-multi-agent-capstone/install.sh`.
- **Step 2 (TMF622 Order Engine):** Implement `evaluate_order_feasibility()` & `remediate_slice_for_order()` in `src/services/order_feasibility.py` (neighbor cell PRB steering).
- **Step 3 (Google ADK & A2A Orchestrator):** Complete `src/agents/adk_a2a_orchestrator.py` (`RANSliceCapacityAgent`, `BSSCommercialSLAAgent`, `TelecomNexusOrchestratorAgent`, `GET /.well-known/agent.json`, `POST /api/v1/a2a`).
- **Step 4 (Local ADK Dev UI):** Run `./scripts/run_adk_web.sh` and test `telecom_nexus` interactively at `http://localhost:8080`.
- **Step 5 (Terraform & Full Verification):** Run `./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run` and `./labs/lab-04-multi-agent-capstone/verify.sh` (**21/21 tests pass!**).

Copy-Paste Prompts, `adk web` & Cloud Commands

- **Prompt 1 — Feasibility Engine:** '/plan Implement evaluate_order_feasibility and remediate_slice_for_order in @src/services/order_feasibility.py per 3GPP TS 28.552 PRB < 80% and neighbor cell steering.'
- **Prompt 2 — ADK + A2A Orchestrator:** '/teamwork Implement RANSliceCapacityAgent, BSSCommercialSLAAgent, TelecomNexusOrchestratorAgent, get_a2a_agent_card, and handle_a2a_jsonrpc_request in @src/agents/adk_a2a_orchestrator.py.'
- **Prompt 3 — Interactive `adk web` (http://localhost:8080):** 'Evaluate TMF622 order ORD-TMF622-9001, execute autonomous 5G slice remediation, and calculate protected ACV.'
- **Cloud Deploy & Verify:** Run `./labs/lab-04-multi-agent-capstone/deploy_gcp.sh --dry-run` and `./labs/lab-04-multi-agent-capstone/verify.sh`.